Weekly Status Report – Week 4

Project: Educational Microcontroller Simulator with Process Scheduling
Technology: Java **Target:** STC89C52
Week: 4 **Date:** 10-17-2026

1. Week 4 Objective

The objective for Week 4 was to enhance the existing Week 3 simulator by separating it into three independent processes (UI, Core and Logging) and enabling communication between them through an IPC mechanism, followed by testing, integration and a comparison of the standalone and multi-process versions.

2. Work Planned

- Separate the existing simulator into UI, Core and Logging processes.
- Use threads where appropriate within the processes.
- Select an IPC mechanism and document the reason for the selection.
- Define and implement communication between the processes.
- Prepare the IPC architecture diagram and IPC test cases.
- Integrate the three processes and perform system testing.
- Compare the standalone and multi-process simulators (execution time, IPC overhead, CPU usage, memory usage).
- Update the GitHub documentation.

3. Work Allocation

Member	Role	Assigned Work
Shahala Shahzeen	Student 1	UI Process: SimulatorUI and the RemoteCPU adapter that connects the UI to the Core.
Neil Saldanha	Student 2	Core Process: CPU execution, Memory, Stack and FIFO Queue behind a socket command server.
Thrupthi Anchan	Student 3	Logging Process: receives log messages and writes them to execution.log.
Aman Assadi	Team Leader	Integration: IntegrationLauncher, IPC integration test, system testing, benchmarking and documentation.

4. Progress / Status

The existing Week 3 CPU, Memory, Stack, FIFO Queue, Assembler and Swing UI were kept and separated into three Java processes. The UI process connects to the Core over a TCP socket on port 5000, and the Core connects to the Logging process over a TCP socket on port 5001. The IntegrationLauncher starts the Logger, Core and UI as separate processes.

Java TCP loopback sockets were selected as the IPC mechanism. The UI sends text commands (PING, LOADSRC, STEP, RUN, RESET, QUIT) and receives state snapshots. The Core sends INFO and ERROR messages to the Logger, which timestamps them and appends them to execution.log. The Core and Logger handle each connection on a thread from an ExecutorService thread pool.

5. Testing and Validation

Test	Result
Build of all sources with javac	Pass
UI/Core socket protocol (PING / PONG)	Pass
Core executes the Week 3 CPU: MOV A,#05 / INC A / HALT gives A=06, PC=2, SP=7, cycles=3	Pass
Core reports HALTED after the program ends	Pass
Core to Logger: execution.log is created and contains the execution message	Pass
Swing SimulatorUI launched under a virtual display and rendered	Pass
Overall IPCIntegrationTest (ALL WEEK 4 IPC TESTS PASSED)	Pass

6. Open Items

- **Benchmark:** the standalone vs. multi-process measurements (execution time, IPC overhead, CPU usage, memory usage) have not been recorded yet. BENCHMARK_TEMPLATE.md is ready to be filled in.
- **POSIX IPC:** TCP sockets are not a POSIX IPC primitive such as FIFO, shared memory or message queues. The team needs to confirm with the instructor whether sockets are acceptable. A POSIX FIFO version for Linux/WSL is the fallback.
- **Performance analysis** will be written after the benchmark values are recorded.

7. Next Steps

The team will record the benchmark values using the same assembly program and machine for both versions, write the performance analysis, confirm the IPC mechanism requirement, and update the GitHub documentation.